

Web API Answers

1. Concept of RESTful Web Service, Web API & Microservice

- **RESTful Web Service:** A web service that adheres to the principles of REST (Representational State Transfer) architecture. It uses standard HTTP protocols (GET, POST, PUT, DELETE) to communicate over the web, treating resources as URIs.
- **Web API:** An Application Programming Interface over the web. In the context of .NET, it is a framework designed to build HTTP services that can be consumed by a broad range of clients, including browsers, mobiles, and desktop applications.
- **Microservice:** An architectural style where a large application is built as a suite of modular, independently deployable, and scalable services. Each microservice runs a unique process and communicates through a well-defined mechanism, often a RESTful Web API.

2. Features of REST Architecture

- **Representational State Transfer (REST):** Resources (data/objects) are manipulated using their representations (e.g., JSON or XML). When a client requests a resource, the server transfers its current state representation.
- **Stateless:** Each request from a client to server must contain all the information needed to understand and complete the request. The server does not store any session state about the client.
- **Messages:** Communication is handled through standard HTTP messages containing headers, URLs, and body data.
- **WebService vs WebAPI:** Traditional Web Services (like SOAP/WCF) are heavyweight, strictly adhere to a contract (WSDL), and only return XML. Web APIs are lightweight, built on standard HTTP, and can return multiple data formats based on the `Accept` header.
- **Not restricted to XML:** Unlike SOAP, REST APIs can respond with JSON (the most common), BSON, plain text, HTML, or custom formats.

3. HttpRequest & HttpResponse

- **HttpRequest:** The message sent by a client (e.g., a browser or mobile app) to a server to initiate an action. It contains the HTTP method (verb), the URI of the resource, HTTP headers (metadata), and optionally a body (payload).

- **HttpResponse:** The message sent back by the server to the client. It contains an HTTP status code indicating success or failure, response headers, and the requested data in the body.

4. Action Verbs

Action verbs (HTTP methods) define the operation to be performed on a resource. They are mapped using attributes in Web API:

- **HttpGet:** Used to read or retrieve a representation of a resource. Declared using `[HttpGet]`.
- **HttpPost:** Used to create a new resource. Declared using `[HttpPost]`.
- **HttpPut:** Used to update/modify an existing resource. Declared using `[HttpPut]`.
- **HttpDelete:** Used to delete a resource. Declared using `[HttpDelete]`.

5. HttpStatusCodes used in WebAPI

- **Ok (200):** The request was successful. Returned via the `Ok()` method.
- **BadRequest (400):** The server could not understand the request due to invalid syntax or validation failure. Returned via the `BadRequest()` method.
- **Unauthorized (401):** The client must authenticate itself to get the requested response. Returned via the `Unauthorized()` method.
- **InternalServerError (500):** The server encountered an unexpected condition that prevented it from fulfilling the request. Returned via the `StatusCode(500)` or `InternalServerError()` method.

6. Simple WebAPI Creation (Read/Write Actions)

A simple Web API controller requires inheriting from the correct base class and using routing attributes.

```
[ApiController] [Route("api/[controller]")] public class ProductsController : ControllerBase { // READ action [HttpGet] public IActionResult Get() { return Ok(new string[] { "Product1", "Product2" }); } IActionResult Post([FromBody] string product) { Ok("Product added successfully"); } }
```

7. Configuration Files of WebAPI

- **Startup.cs (.NET Core):** The entry point for configuration. Contains `ConfigureServices` for configuring dependency injection and `Configure` for setting up the HTTP request pipeline (middleware).

- **appSettings.json:** Replaces `web.config`. Used to store application configuration settings like database connection strings, API keys, and logging settings.
- **launchSettings.json:** Used locally by developers to configure how the application is launched (e.g., environment variables, application URL, IIS Express vs Kestrel).
- **RouteConfig & WebApiConfig (.NET 4.5):** In older MVC applications, `RouteConfig.cs` maps MVC web routes, while `WebApiConfig.cs` configures specific Web API routes (typically prefixed with `api/`).

8. First Web API using .NET Core

Creation Steps:

1. Create a new project in Visual Studio and select "ASP.NET Core Web API".
2. Ensure the template is set to create controllers (uncheck minimal APIs if applicable) and check options for read/write actions.
3. Notice the creation of a default Controller (e.g., `ValuesController` or `WeatherForecastController`). It automatically inherits from `ControllerBase` and is decorated with `[ApiController]`.
4. The controller will have predefined action methods mapped to GET, POST, PUT, and DELETE action verbs.

Execution:

In the browser or a tool like Swagger/Postman will launch. Navigating to the endpoint (e.g., `https://localhost:port/api/values`) triggers the GET action method, returning the expected JSON response payload.

```
[
  {
    "date": "2026-06-25T10:30:00.000Z",
    "temperatureC": 25,
    "temperatureF": 76,
    "summary": "Mild"
  },
  {
    "date": "2026-06-26T10:30:00.000Z",
    "temperatureC": 32,
    "temperatureF": 89,
    "summary": "Hot"
  },
  {
    "date": "2026-06-27T10:30:00.000Z",
    "temperatureC": 15,
    "temperatureF": 58,
    "summary": "Chilly"
  }
]
```